Exercises

Exercise 7: Reproducible reports with R markdown

Read Chapter 8 to help you complete the questions in this exercise.

Before the session. There are two things to sort out beforehand, and the second one takes a few minutes, so please don't leave it until the morning of the session.

First, you need the `rmarkdown` and `knitr` packages. Both normally come with RStudio, but check by running `library(rmarkdown)` and `library(knitr)` in the console. If either gives an error, install it with `install.packages("rmarkdown")`. See Appendix A of the Introduction to R book if you need more detail.

Second, you'll be producing PDF documents in this exercise, which is what your assessment needs, and that requires a LaTeX installation. You almost certainly don't have one, and you don't need to know anything about LaTeX to use it. Run these two lines in the console:

```
install.packages("tinytex")
tinytex::install_tinytex()
```

The second line downloads a few hundred megabytes, so give it time and a decent connection, and restart RStudio afterwards. Check it worked by running `tinytex::is_tinytex()`, which should return `TRUE`.

If you run into real problems we'll help you at the start of the session, but please try beforehand so we don't lose teaching time to installing software.

Up to now everything you've written has been code, for your own use. This exercise is about producing a *document* for somebody else to read, in which your writing, your code and your results all live in the same file. Because they're in the same file they can't drift apart, which is what we mean by a reproducible report. Change the data, press one button, and every number, figure and table updates itself.

You'll build **one document** across the whole exercise, adding something to it at each question, so don't start a new file each time. By the end you'll have a short report on the cardiac data and a template you can reuse for your assessment. There's a lot of new vocabulary here, so take it slowly and knit often. Knitting after every small change is the quickest way to see what each thing does, and the quickest way to find out which change broke something.

1. Open the RStudio Project you have been using for this course. Create a new R markdown document by going to `File` -> `New File` -> `R Markdown...`.

The first time you do this, RStudio may tell you it needs to install some packages. Let it, and wait for it to finish.

In the dialogue box that appears, type `Cardiac study report` in the Title box, put your name in the Author box, select **PDF** on the left, and click OK. PDF is the format your assessment has to be submitted in, so it is the one worth practising. RStudio opens a new document that's already full of example content about cars. That's normal, and we'll delete it in a moment.

Before you change anything at all, save the file into your Project directory as `cardiac_report.Rmd` (`File -> Save As...`), then click the **Knit** button at the top of the editor. You can also press Ctrl+Shift+K, or Cmd+Shift+K on a Mac.

Do this before you understand any of it. If a PDF appears then your setup is working, and you can be confident that anything which goes wrong later is something you can fix.

2. Look at the document that appeared alongside the file that produced it. An R markdown file has only three kinds of content, and everything else is a variation on them:

- the **YAML header** at the very top, between the two lines of `---`, which controls the document as a whole
- **text**, written in a simple markup language called markdown
- **code chunks**, which look like this:

````
```{r}
mean(cardiac$age)
```
````

Find one of each in your file. Then delete everything below the first code chunk, the one called `setup`, leaving the YAML header and that chunk in place. Knit again to check you have an empty but working document. This is your starting point.

3. Now take control of the YAML header. At the moment it has a title, an author and a date, and an `output` line saying `pdf_document`. Replace that line with the four shown below.

Two warnings, because this is where people lose ten minutes. Indentation in YAML matters, so copy the spacing exactly. And use spaces, never tabs.

```yaml
---
title: "Cardiac study report"
author: "Your Name"
date: "20 August 2026"
output:
  pdf_document:
    toc: true
    number_sections: true
---
```

Add `toc: true` first and knit. Then add `number_sections: true` and knit again. Adding them one at a time, and knitting after each, is how you see what each one actually does.

4. Time to write something. Underneath the setup chunk, write a short introduction to the cardiac study using markdown formatting only, with no code yet. Include:

- a heading, written as `# My heading`
- a second, smaller heading below it, written as `## My smaller heading`

- a sentence with a word in **bold**, written as `**bold**`, and a word in *italics*, written as `*italics*`
- a bulleted list of four of the variables in the dataset, each line starting with a dash and a space

That is enough to be going on with. If you want to add a link, a numbered list, a quotation or a table, Section 8.5.2 of the book has the syntax for all of them, and there is a summary on the R markdown cheat sheet under `Help -> Cheat Sheets`.

Knit, and put your source file and the output side by side. Notice that the headings you wrote have appeared in the table of contents on their own, because of the `toc: true` you added in Q3.

5. Now add some code. Insert a new code chunk using the **Insert** button at the top right of the editor and choosing R, or with the keyboard shortcut Ctrl+Alt+I (Cmd+Option+I on a Mac). An empty chunk appears.

Give the chunk a name by typing it straight after the `r`, so the first line reads ```` ```{r import} ````. Naming chunks costs nothing and makes them far easier to find when something goes wrong.

Inside the chunk, import `cardiacdata.txt` exactly as you did in **Exercise 3, Q5**: with the `read.table()` function, writing out the `header`, `sep` and `stringsAsFactors` arguments, and assigning the result to `cardiac`. In the same chunk, add the two `factor()` lines from **Exercise 3, Q7** that create `Fsex` and `Fsmoking`, since you'll want them later.

Then insert a second chunk, name it `structure`, and put `str(cardiac)` in it.

Run each chunk in the editor with the small green arrow at its top right, then knit the whole document. Notice that knitting starts a completely fresh R session: anything you made earlier in the console does not exist when the document knits, which is exactly what makes the document reproducible. It also means the chunk that imports the data has to come before any chunk that uses it.

6. Everything so far has been about the mechanics of the document. This question is about what a report is actually for, and it's the one that most resembles what you'll be asked to do in your assessment. A reader needs to know not just what you found, but what you did to the data before you found it, and why.

Add a section to your report called `## Data validation`, immediately below your `structure` chunk. In it, put a chunk named `validation` holding the three lines from **Exercise 4, Q1** that set the impossible `bmi`, `triglyceride` and `hdlchol` values to `NA`. Remember that your report imports the raw file from scratch every time it knits, so if the cleaning isn't in the document then it hasn't happened.

Then write a short paragraph underneath, two or three sentences, saying which variables you changed, how many values in each, and why you set them to `NA` rather than deleting those patients or correcting the numbers yourself. Type the counts in by hand for now. You'll come back to them in Q9, and there's a wrinkle waiting for you there.

Give this chunk `echo = TRUE` when you get to Q8. A section that tells the reader you cleaned the data but hides how is not much use to anybody.

Now do the same for a transformation. Underneath, add a chunk named `alcohol-transform` that creates the square root of `alcohol` as you did in **Exercise 5, Q9**, and draws the histograms of `alcohol` and `sqrt(alcohol)` side by side. Give it a caption with `fig.cap`, and write a sentence or two saying why you transformed it.

7. Add a figure. Insert another chunk, name it `chol-plot`, and use it to redraw the boxplot of HDL cholesterol by smoking status from **Exercise 5, Q11**. You already have `Fsmoking` from your import chunk, so this is just the `boxplot()` call.

Now give the figure a caption. Chunk options go inside the curly braces, after the chunk name, separated by commas. The first line of your chunk should read:

```
```{r chol-plot, fig.cap = 'HDL cholesterol by smoking status.'}
```
```

Knit and find your caption underneath the plot. Then add `fig.width = 4` to the same line, knit again, and see what happens to the size of the text relative to the plot.

There are a great many other chunk options for figures, controlling height, alignment, resolution and so on. You do not need them today. Chapter 8 of the book lists the ones worth knowing.

8. Now think about who is going to read this document. Somebody checking your working will want to see your code, but a supervisor, a manager or an examiner usually won't - they want the results, and the code just gets in the way. R markdown lets you decide this chunk by chunk with the `echo` option.

a) Add `echo = FALSE` to your `chol-plot` chunk, alongside the caption you added in Q7, and knit. The plot is still there but the code that drew it is not.

b) Now do the same for the whole document rather than one chunk at a time. Go to your `setup` chunk at the top and put this line inside it:

```
knitr::opts_chunk$set(echo = FALSE)
```

Knit. Every chunk now hides its code, so you can delete the `echo = FALSE` you added in part (a) and it will still be hidden.

c) Code isn't the only thing R puts in your document uninvited. Try this. Add `library(vioplot)` on its own line at the top of your `chol-plot` chunk, using the package you installed in Exercise 5, and knit. Even with the code hidden you get several lines of this in the middle of your report:

```
Loading required package: sm
Package 'sm', version 2.2-6.0: type help(sm) for summary information
Loading required package: zoo
```

Those are **messages**. R also produces **warnings**, which appear the same way and look far more alarming to a reader than they usually deserve to. Neither belongs in a finished report. Add `warning = FALSE` and `message = FALSE` to your `opts_chunk$set()` line, separated by commas, and knit again. The messages are gone. Now delete the `library(vioplot)` line, since you don't need it.

d) Finally, make one exception to the rule you have just set. Add `echo = TRUE` to the first line of your `structure` chunk and knit. That one chunk shows its code while the rest do not. Setting a default for the document and then overriding it where you need to is how nearly every real report is put together.

9. Have a look back at the paragraph you wrote in Q6. You typed those numbers in by hand, and hand typed numbers go stale. If the data changes, or you correct something later, your writing quietly stops matching your results and nothing tells you.

R markdown fixes this with **inline code**. In the middle of an ordinary sentence you write a backtick, then the letter `r`, then some R code, then a closing backtick, and when you knit, the answer appears instead. Like this:

```
The study included `r nrow(cardiac)` patients.
```

which comes out as "The study included 163 patients."

Go back to your data validation paragraph and replace every number you typed by hand with inline code. Then add a sentence to your introduction giving the number of patients and their mean age in the same way. Use `round()` on the mean so you don't get six decimal places.

There's one catch worth knowing about now rather than later. Inline code runs in the order it appears in the document, just as chunks do, so it only works if it sits **below** the chunk that made the object it uses. Put a piece of inline code above your `import` chunk and the knit will stop with `object 'cardiac' not found`. If you ever see that message, this is nearly always the reason.

Then test that it really works. Temporarily change your import chunk so it keeps only the first 100 patients, by adding this line at the end of the chunk:

```
cardiac <- head(cardiac, 100)
```

Knit, and watch your sentence update itself. Then delete that line and knit again.

10. Add a table. If you simply print a dataframe in a chunk you get the console output, monospaced and ugly. The `kable()` function from the `knitr` package formats it as a proper table instead.

Insert a chunk, name it `summary-table`, and start by recreating the `aggregate()` summary from **Exercise 4, Q4**: the mean of `age`, `systolic` and `tchol` for each smoking category. Assign it to an object called `smoke_summary`.

Knit and look at how ugly it is. Now put `knitr::kable(smoke_summary)` on the next line, knit again, and compare. Finally add `digits = 1` and a `caption` to the `kable()` call to tidy it up.

11. Not every figure in a report is one that R drew. You might need a diagram, a photograph, a map, or a figure somebody else made. Add the plot you exported in **Exercise 6, Q9**, `ex6_admissions.png`, to your report.

The simplest way needs no chunk at all. Type this straight into your text, changing the path to wherever your file actually is:

```
![Alcohol-related hospital admissions by council area.](output/ex6_admissions.png)
```

An exclamation mark, then the caption in square brackets, then the file path in round brackets. Knit and check the figure appears.

If you no longer have the file, download it here and save it into your `output` directory, the one you created in **Exercise 1, Q12**.

One thing that catches everybody out: the path is relative to where the `.Rmd` file is, not to your working directory. If the image does not appear, the path is almost always the reason.

12. If you have time, here's one of the nicer things about R markdown: the same source file will give you a completely different kind of document without you rewriting a word of it. In your YAML header, change `pdf_document` to `html_document` and knit. You'll get a web page instead, built from exactly the same text, code and figures. While you're there, add `toc_float: true` on its own line underneath `toc: true` and knit again to see the contents float alongside the page as you scroll. That option only works in HTML, so change the header back to `pdf_document` and delete the `toc_float` line before you finish.

Two things worth knowing about, but not now

The visual editor. RStudio can show an R markdown document formatted as you type, more like a word processor, hiding the markdown syntax. You turn it on with the small compass icon at the top right of the editor. It is genuinely useful, especially for tables and links. We have deliberately left it until the end so that you learn what the markup actually is first, because when something goes wrong it is the markup you have to fix.

Quarto. Quarto is the successor to R markdown. It does the same job, works with languages other than R, and its syntax is so close that nearly everything in this exercise transfers unchanged. If you carry on using R after this course you will meet it. Start at quarto.org.

End of Exercise 7